

Bienvenidos a Orientación a **Objetos** 2



Orientación a Objetos 2

Introducción



En las teorías:

- Alejandra Garrido
- Federico Balaguer
- Gabriela Perez

En las explicaciones de práctica:

- Agustín Ortú
- Germán Ruiz
- Leonardo Loza Bonora
- Juan Cruz Gardey

En las consultas de práctica:

Cristian, Diego, Felipe,
Guadalupe, Ignacio,
Luciana, Nero, Catalina,
Martín, Giuliana,
Rafael, Nathaniel, Camila,
Bautista,
Joaquín, Lautaro, Bianca,
Matheo, Augusto

2 Bandas Horarias - todo presencial

- TEORIA
 - Lunes 8:30 a 10:30 hs - Aula 4
 - Lunes 16 a 18 hs. - Aula 5 (en mayo cambia el aula)
- HORARIOS DE PRACTICA
- Comienzan la semana del 16/3
- Explicación de práctica:
 - Lunes 18 a 19 hs: Aula 5
 - Martes 8:30 a 9:30 hs: Aula 10A
- Práctica:
 - 7 horarios de práctica disponibles (2 de mañana y 5 de tarde) donde deberán inscribirse esta semana

Objetos 2 - Aprobación de la materia

- Cursada:
 - se aprueba con un examen parcial, que tiene 2 recuperatorios. El parcial se aprueba por temas.
- Fechas de parcial:
 - 6 de junio
 - 27 de junio
 - 11 de julio
- Final de la materia:
 - Examen final
 - Promoción: se rinde el 11 de julio. Única fecha

Objetos 2 - Promoción

- Para promocionar la materia se requiere:
 - aprobar la cursada en 1ra o 2da fecha de parcial;
 - responder correctamente las preguntas extra que se agregan al parcial para acceder a la promoción;
- aprobar un examen integrador de promoción que se rinde por única vez el 11/7/26 con calificación 6 o más.

- Todo el material estará disponible en la plataforma Cátedras
- Pero... no se queden solo con nuestras transparencias, es importante que estudien de los libros obligatorios de la materia:
 - Design Patterns. Elements of Reusable Objects Oriented Software. Gamma, Helm, Johnson, Vlissides, Addison-Wesley.
 - Refactoring: Improving the Design of Existing Code. Fowler, Martin. Addison-Wesley. **1ra edición**
 - Refactoring to Patterns. Joshua Kerievsky. Addison Wesley.
 - Implementing Application Frameworks: Object-Oriented Frameworks at Work. Fayad, Schmidt, Johnson (*eds*).

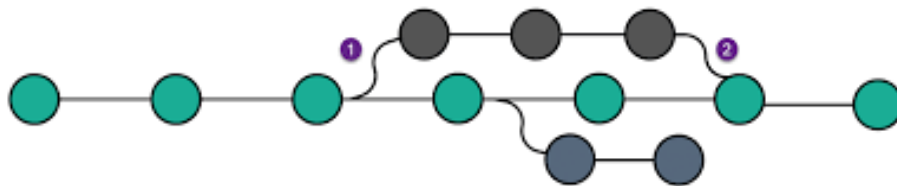
- **Métodos ágiles**

- se desarrolla en iteraciones
- cada iteración extiende código existente con nueva funcionalidad
- poca documentación
- testing automático e integración continua



- Arquitecturas complejas (arquitecturas modulares, capas con distintos concerns: interfaz de usuario, persistencia, seguridad, ...), arquitecturas orientadas a servicios (conexión con código ajeno, desarrollo de APIs, ...)
- El cambio es continuo, el crecimiento es continuo, la complejidad es cada vez mayor

Contexto: herramientas para el desarrollo



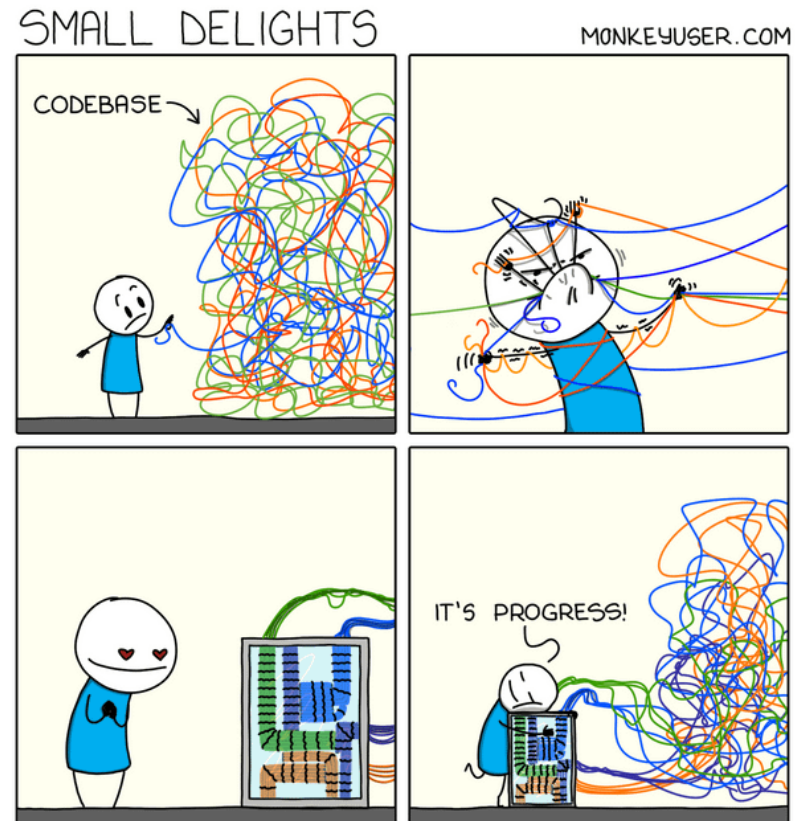
- **Version Control Systems: Branching & merging, DevOps**
 - qué implica?
 - entender código existente (50% del tiempo de mantenimiento)
 - extender código existente
 - code review antes de cada merge
 - entrega continua de alta calidad
- **IA Generativa: Copilot, Cursor, Claude, Gemini, etc.**
 - qué implica?
 - una gran ayuda para tareas rudimentarias
 - necesitamos desarrollar pensamiento crítico

- Desarrollar **criterio para distinguir calidad** en el diseño de software
- Adquirir buenas prácticas en el proceso de **mejora de la calidad** del software
- Desarrollar la capacidad de cambiar el software de manera **rápida y sustentable**
 - consciente, evitando el desperdicio de recursos, reconociendo oportunidades, responsable, con calidad, buscando la mejora continua

Qué vamos a aprender en Objetos 2

- Reconocer las fuerzas que llevan al deterioro de la arquitectura y aprender a reconocer oportunidades para mejorarla
- Con el objetivo de lograr código legible, extensible, mantenible
- En cada iteración
- Paso a paso

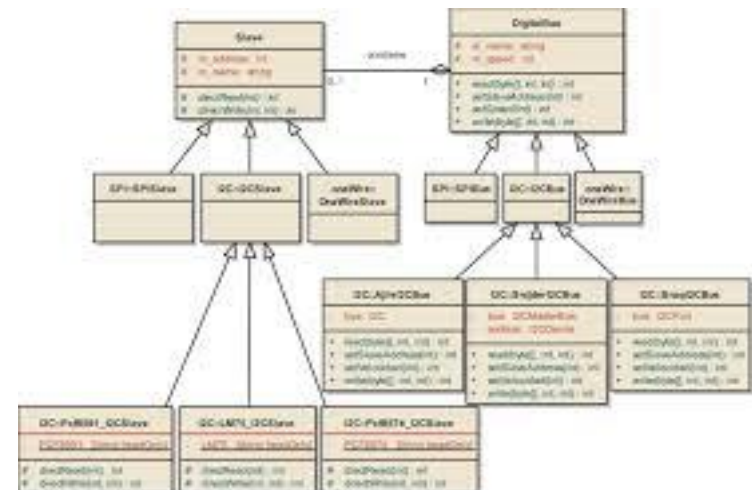
REFACTORING.



Qué más vamos a aprender en Objetos 2

- Construir diseños reusables, más sensibles a la evolución
- Reconocer las consecuencias positivas y negativas de un diseño (fundamentar y criticar diseños)
- Hacer evolucionar diseños, aplicando mejoras paso a paso

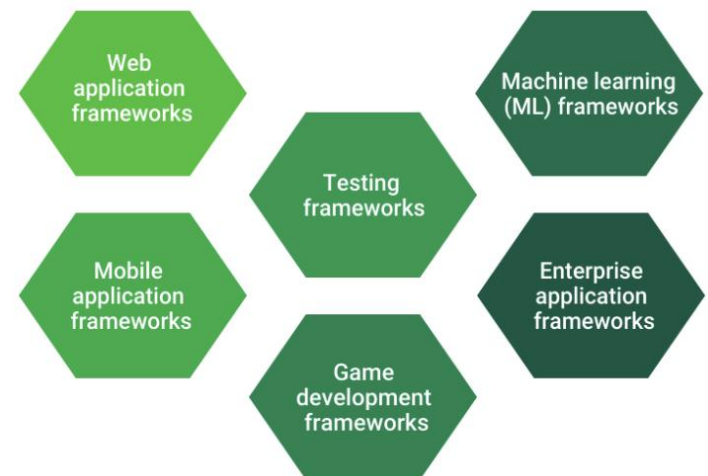
PATRONES DE DISEÑO



Qué otra cosa vamos a aprender en Objetos 2

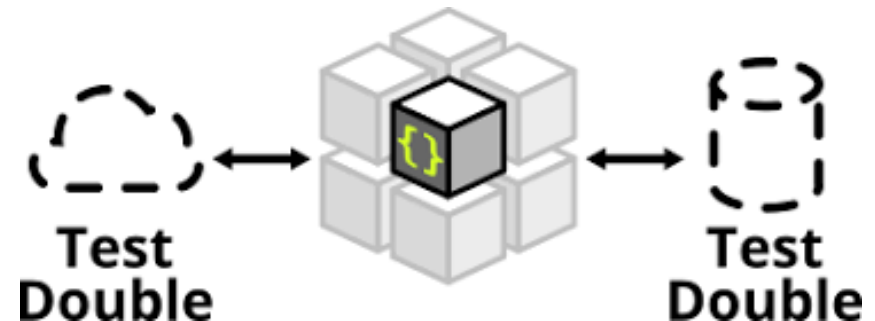
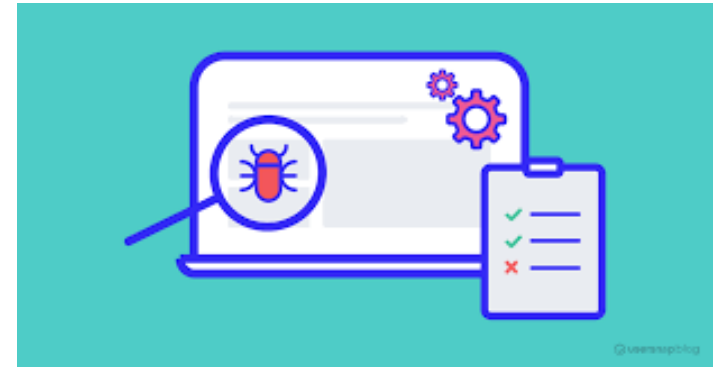
- Instanciar arquitecturas reusables para una familia de aplicaciones, reconociendo puntos de variabilidad y extensión de estas arquitecturas
- Entender las implicancias de utilizar estas arquitecturas
- Diseñar y programar estas arquitecturas

FRAMEWORKS.



Buscando calidad en lo que hacemos

- Importancia del testing
- Tipos de Tests
- Test Driven Development
- Estrategias de testing con test doubles



- Es muy importante tener la materia al día
- Van a tener el material en Cátedras, para que les sirva de repaso. Las transparencias pueden estar incompletas y no tienen la función de reemplazar las teorías y explicaciones en el aula
- Es fundamental:
 - participar de las teorías y las explicaciones de práctica
 - leer la bibliografía que se les indica para cada tema
 - hacer los ejercicios de práctica y venir a las consultas a presentar y fundamentar sus diseños